

```

1  #include <stdio.h>
2  #define length 100
3  typedef struct {
4      int data[length];
5      int size;
6  }List;
7
8  void makenull_List(List *L) {
9      L->size = 0;
10 }
11
12 int empty_List(List *L) {
13     return L->size==0;
14 }
15
16 int element_at(List *L, int i) {
17     return L->data[i-1];
18 }
19
20 void push_back(List *L, int x) {
21     if (L->size+1 == length)
22         return;
23     L->data[L->size] = x;
24     L->size++;
25 }
26
27 void copy_List(List *L1, List *L2) {
28     makenull_List(L1);
29     if (empty_List(L2))
30         return;
31     int i;
32     for (i=1; i<=L2->size; i++)
33         push_back(L1, element_at(L2, i));
34 }
35
36 #define MAX_N 100
37 typedef struct {
38     int A[MAX_N][MAX_N];
39     int n, m;
40 }Graph;
41
42 void init_graph(Graph *G, int n) {
43     G->n=n;
44     G->m=0;
45     int u, v;
46     for (u=1; u<=n; u++)
47         for (v=1; v<=n; v++)
48             G->A[u][v] = 0;
49 }
50
51 void add_edge(Graph *G, int u, int v) {
52     G->A[u][v] = 1;
53     G->m++;
54 }

```

```

55
56 int r[MAX_N];
57
58 void rank(Graph *G) {
59     int u, v, d[MAX_N];
60     for (u=1; u<=G->n; u++) {
61         d[u] = 0;
62         for (v=1; v<=G->n; v++)
63             if (G->A[v][u] > 0)
64                 d[u]++;
65     }
66
67     List L1, L2;
68     makenull_List(&L1);
69     for (u=1; u<=G->n; u++)
70         if (d[u]==0)
71             push_back(&L1, u);
72
73     int k = 0, i;
74     while (!empty_List(&L1)) {
75         makenull_List(&L2);
76         for (i=1; i<=L1.size; i++) {
77             u = element_at(&L1, i);
78             r[u] = k;
79
80             for (v=1; v<=G->n; v++)
81                 if (G->A[u][v] > 0) {
82                     d[v]--;
83                     if (d[v]==0)
84                         push_back(&L2, v);
85                 }
86         }
87         copy_List(&L1, &L2);
88         k++;
89     }
90 }
91
92 int main() {
93     Graph G;
94     int n, m, u, v, e;
95     FILE *f = fopen("dothixephang.txt", "r");
96     fscanf(f, "%d%d", &n, &m);
97     init_graph(&G, n);
98     for (e=1; e<=m; e++) {
99         fscanf(f, "%d%d", &u, &v);
100         add_edge(&G, u, v);
101     }
102
103     rank(&G);
104
105     printf("Xep hang dinh cua do thi:\n");
106     for (u=1; u<=G.n; u++)
107         printf("rank[%d] = %d\n", u, r[u]);
108
109     fclose(f);
110     return 0;
111 }

```